TP3 (parte 1)

Integrantes

- Antonino, Tadeo - tadeo.antonino@mi.unc.edu.ar
- Fioramonti, Martino - martino.fioramonti@mi.unc.edu.ar
- Quintana, Ignacio Agustin - ignacio.agustin.quintana@mi.unc.edu.ar

---

Repositorio GitHub

https://github.com/IgnacioQuintana57/EN_MI_PC_FUNCIONA

Introducción

En este TP ejecutaremos un trozo de código que configura nuestro procesador para llevarlo desde el modo real al modo protegido.

Los procesadores x86 mantienen compatibilidad con sus antecesores y para agregar nuevas funcionalidades deben ir “evolucionando” en el tiempo durante el proceso de arranque. Todos los CPUs x86 comienzan en modo real en el momento de carga (boot time) para asegurar compatibilidad hacia atrás, en cuanto se los energiza se comportan de manera muy primitiva, luego mediante comandos se los hace evolucionar hasta poder obtener la máxima cantidad de prestaciones posibles. El modo protegido es un modo operacional de los CPUs compatibles x86 de la serie 80286 y posteriores. Este modo es el primer salto evolutivo de los x86. El modo protegido tiene un número de nuevas características diseñadas para mejorar la multitarea y la estabilidad del sistema, tales como la protección de memoria, y soporte de hardware para memoria virtual como también la conmutación de tareas.

Desafío UEFI y coreboot

¿Qué es UEFI, cómo usarlo y qué función se puede llamar?

UEFI (Unified Extensible Firmware Interface) es el estándar de firmware que reemplazó a la BIOS. Básicamente, es como un sistema operativo que hace de puente entre el hardware de una computadora y el sistema operativo (LINUX ❤️, Windows 🤢, etc). Es el encargado de inicializar los componentes, verificar firmas de seguridad (Secure Boot) y cargar el bootloader.

¿Cómo usarlo? EXisten dos formas principales de interactuar con UEFI:

Como usuario/administrador: Apretando una tecla (F2, Del, F12) al prender la PC para acceder a su interfaz gráfica. Desde ahí configuras el orden de arranque, activas/desactivas Secure Boot, haces overclocking o gestionas contraseñas de hardware.

Como desarrollador: Podes escribir aplicaciones UEFI (archivos .efi, programados generalmente en C) que se ejecutan antes de que cargue el sistema operativo, o usar la terminal integrada UEFI Shell.

Ejemplo de una función: A nivel de código, UEFI provee “Servicios de Arranque” (Boot Services) y “Servicios en Tiempo de Ejecución” (Runtime Services). Una función clásica a la que un programador o el sistema operativo puede llamar es GetVariable(). Esta función permite leer información directamente de la memoria no volátil (NVRAM) de la motherboard.

Bugs de UEFI que pueden ser explotados

El firmware es un objetivo muy importante para los atacantes (crackers) porque el código que se ejecuta ahí tiene más privilegios que el propio sistema operativo y sobrevive a formateos del almacenamiento. Algunos casos reales y famosos de vulnerabilidades UEFI son:

LogoFAIL (Descubierto en 2023): Una vulnerabilidad insólita donde el error estaba en cómo la UEFI procesaba las imágenes (el logo de la marca de la PC que ves al encenderla). Un atacante podía reemplazar

ese logo por una imagen maliciosa que provocaba un desbordamiento de búfer y ejecutaba código antes de que iniciara Windows o Linux, esquivando todas las defensas.

BlackLotus (2022/2023): Fue el primer bootkit en estado salvaje (wild) capaz de eludir el Secure Boot de UEFI (explotando la vulnerabilidad CVE-2022-21894). Podía desactivar Windows Defender y otras herramientas de seguridad desde la raíz, garantizando persistencia total.

LoJax (2018): Descubierto por ESET, fue el primer rootkit UEFI utilizado en ataques reales (asociado al grupo ruso APT28). Modificaba la memoria flash SPI de la placa madre. Aunque cambiaras el disco duro completo, el malware seguía infectando la PC en cada inicio.

¿Qué son CSME e Intel MEBx?

Estas tecnologías son, en esencia, una computadora dentro de la computadora.

CSME (Converged Security and Management Engine): Es un microcontrolador y un subsistema completamente aislado e incrustado en los chipsets de Intel. Tiene su propio procesador, su propia memoria y corre su propio sistema operativo (basado en MINIX). Funciona "fuera de banda" (Out-of-Band), lo que significa que está encendido y conectado a la red incluso si la PC está apagada o no tiene sistema operativo. Se encarga de tareas criptográficas (TPM), gestión de derechos digitales (DRM) y administración remota corporativa.

Intel MEBx (Management Engine BIOS Extension): Es el menú de configuración de ese subsistema. Es una extensión a la que puedes acceder durante el arranque de la computadora (generalmente presionando Ctrl+P). Los administradores de sistemas (IT) usan el MEBx para habilitar o deshabilitar la administración remota (Intel AMT), configurar contraseñas de red y asignar direcciones IP a este procesador oculto.

Coreboot: ¿Qué es, quién lo usa y sus ventajas?

Coreboot es un proyecto de firmware de código abierto respaldado por la Free Software Foundation. Nació con la filosofía opuesta a UEFI: en lugar de ser un sistema gigante y complejo, Coreboot busca ser lo más minimalista posible. Su único trabajo es inicializar el procesador, la

memoria y el hardware básico extremadamente rápido, y luego pasarle el control inmediatamente a un “payload” (como SeaBIOS para emular una BIOS antigua, o TianoCore para emular UEFI, o incluso un kernel de Linux directo).

¿Qué productos lo incorporan?

Chromebooks: Casi todos los equipos con Google ChromeOS (fabricados por HP, Acer, Asus, etc.) usan Coreboot por debajo.

Notebooks enfocadas en privacidad/Linux: Marcas como System76, Purism y Star Labs.

Routers y hardware embebido: Muchos dispositivos de red comerciales usan Coreboot por su ligereza.

Ventajas de su utilización:

MAyor velocidad: Como no carga drivers innecesarios (ni soporta ratón en 3D, ni logos pesados como UEFI), los tiempos de arranque pasan de varios segundos a apenas milisegundos.

Transparencia y Seguridad (Open Source): El código es público. Los desarrolladores pueden auditarlo para asegurarse de que no haya puertas traseras del fabricante y pueden eliminar módulos (como desactivar parte del Intel ME) para reducir la “superficie de ataque”.

Libertad del usuario: No estás atado a las actualizaciones de la placa madre del fabricante (que suelen abandonar el soporte en un par de años). La comunidad puede seguir actualizando el firmware del equipo.

Desafío Linker

¿Qué es un linker? ¿Qué hace?

El linker es un programa fundamental en la cadena de compilación. Cuando se compila código fuente, el compilador no genera un ejecutable listo para usar, genera archivos objeto (archivos .o).

Resuelve referencias (Símbolos): Si en archivo_A.c llamas a una función que está en archivo_B.c, el compilador no sabe dónde está esa función. El linker junta ambos archivos y conecta esa llamada con la ubicación real de la función.

Combina secciones: Agrupa todo el código ejecutable de tus archivos en una sección (suele llamarse .text), todas tus variables globales en otra (.data o .bss), etc.

Asigna direcciones de memoria: Le da a cada instrucción y variable una dirección de memoria final para que el procesador sepa dónde encontrarlas cuando el programa se ejecute.

¿Qué es la dirección que aparece en el script del linker? ¿Por qué es necesaria?

En un script de linker (archivo .ld), sueles ver una línea que asigna un valor al "Location Counter" (representado por un punto .), por ejemplo: . = 0x7C00; o . = 0x100000;.

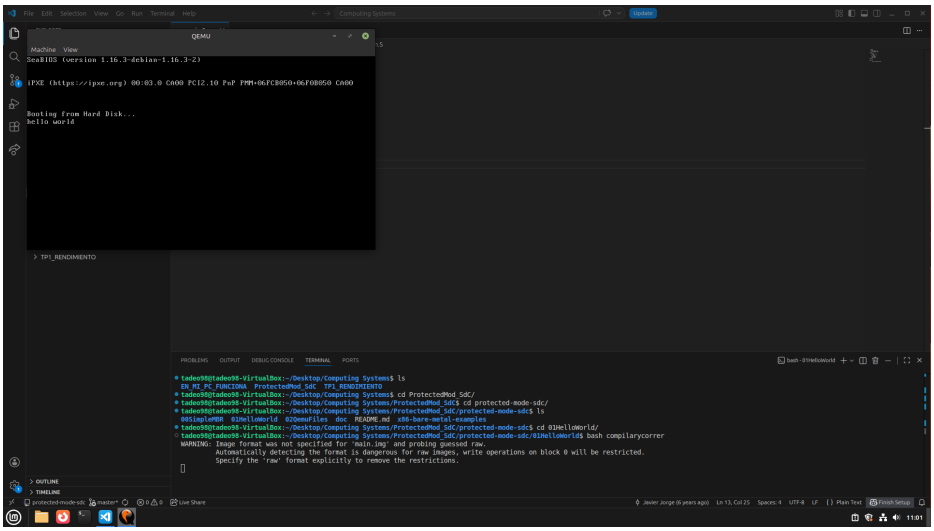
Qué es: Es la dirección de memoria física (o virtual) donde tu programa espera ser cargado en la RAM al momento de ejecutarse.

Por qué es necesaria: En programación normal, el sistema operativo (Windows/Linux) carga tu programa en cualquier parte libre de la RAM y ajusta las direcciones por ti. Pero en bare-metal no hay sistema operativo. La BIOS o UEFI carga tu código en un lugar específico de la memoria (por ejemplo, los bootloaders clásicos siempre se cargan en 0x7C00). El linker necesita saber esto para calcular correctamente los saltos (jumps) y dónde buscar las variables. Si le mientes al linker, tu programa buscará cosas en la memoria equivocada y colapsará.

Booteable Hello World

Para esta actividad es necesario contar con una PC/Notebook lo suficientemente antigua como para que sea compatible con el booteo BIOS solicitado, y nosotros no tenemos equipos de esas características.

Lo que pudimos hacer es emular esto mediante QEMU como se ve en la imagen siguiente:



BooteoQEMU

¿Para qué se utiliza –oformat binary?

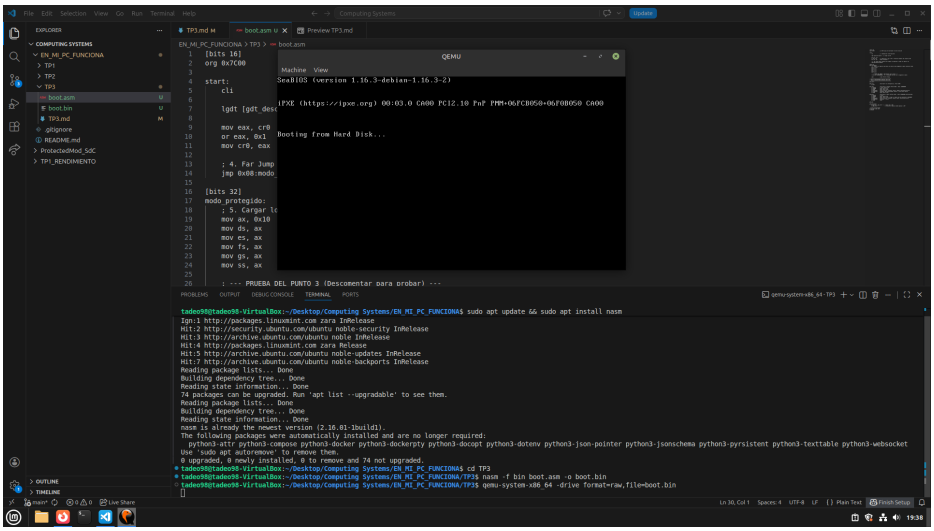
Se utiliza para pedirle al linker que elimine todas las cabeceras, tablas y metadatos del archivo final, y en su lugar genere un 'flat binary'. Este binario contiene única y exclusivamente las instrucciones de máquina crudas y los datos estáticos, sin ningún tipo de formato o envoltura.

Si no usaramos --oformat binary y le pasamos a la BIOS un archivo ELF normal, el procesador tomaría el texto de la cabecera (metadatos) e intentaría "ejecutarlos" como si fueran comandos. Como resultado, la computadora colapsaría o se reiniciaría instantáneamente.

Desafío Modo Protegido

Se nos pidió hacer un código en assembler que pase a modo protegido. El programa al iniciar configura el procesador para saltar al Modo Protegido de 32 bits, carga la tabla de memoria (GDT) y entra en un bucle infinito.

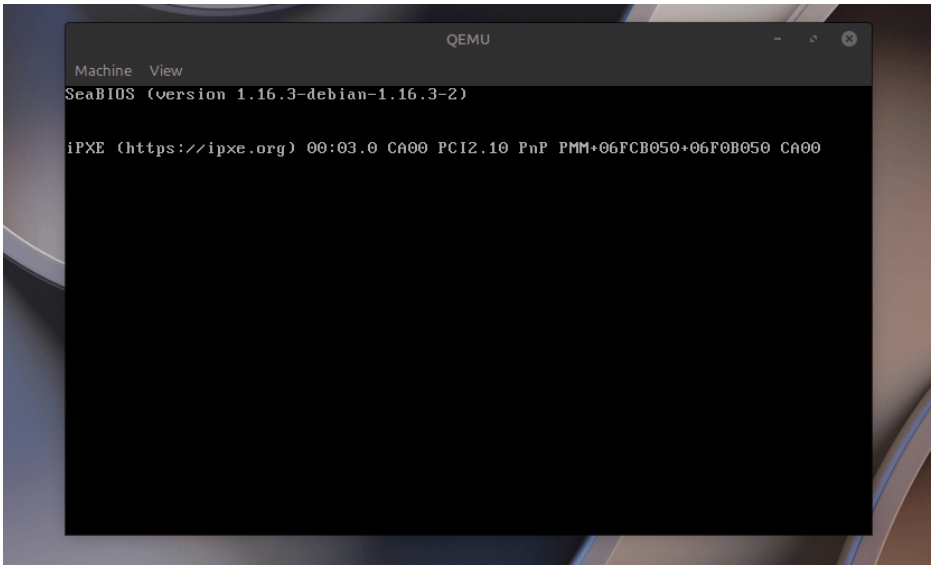
El sistema operativo bare-metal es estable. QEMU se queda “congelado” esperando instrucciones para siempre como se observa en la siguiente imagen:



QEMU

El programa entra a Modo Protegido igual que antes, pero justo antes de entrar al bucle infinito, intenta escribir una letra ‘A’ en la memoria.

Como ahora modificamos la GDT para que la memoria sea estrictamente de “Solo Lectura”, el procesador intercepta la escritura, detecta una violación de seguridad y lanza un error. Al no saber cómo manejar el error, la CPU entra en pánico (Fallo Triple) y reinicia la máquina a la fuerza. QEMU parpadea infinitamente en un ciclo de reinicios como se observa en el gif:



QEMU

En Modo Protegido, los registros de segmento (como CS, DS, ES) ya no se cargan con las direcciones base físicas de la memoria (como se hacía en Modo Real multiplicando por 16). En su lugar, se cargan con Selectores de Segmento (Segment Selectors). En nuestro código, CS se carga con el valor 0x08 y los registros de datos con 0x10.

¿Por qué? Porque la información que describe a un segmento en Modo Protegido (dirección base de 32 bits, límite, permisos de lectura/escritura, y niveles de privilegio) ocupa 8 bytes por entrada en la GDT. Esta cantidad de información es demasiado grande para caber en un registro de segmento de 16 bits. Por lo tanto, los valores como 0x08 o 0x10 actúan simplemente como un índice o puntero. Le indican al procesador en qué desplazamiento exacto (offset) dentro de la GDT debe buscar el descriptor correspondiente para aplicar las reglas de memoria y seguridad antes de ejecutar una instrucción.